

Machine Learning Training Handbook

A beginner-friendly, self-guided 5-week preparation program for data science internship candidates at IDX Exchange

Version 3.0

Focus: California residential sold data • Target variable: ClosePrice • Final model: XGBoost

Who this handbook is for

This handbook assumes little to no prior machine learning experience. It starts with a mental model of how house prices work, then moves through local setup, data cleaning, feature engineering, visualization, model evaluation, and a final XGBoost prediction workflow.

Program at a Glance

Phase	Theme	Main Outcome
Week 0	Setup and orientation	Your environment works and you can load the CSV files safely.
Week 1	Data understanding and cleaning	You can combine files, inspect columns, filter to the right homes, and prepare a training table.
Week 2	Feature engineering and visualization	You build useful features, understand why each was chosen, handle nulls systematically, and explore the data visually.
Week 3	Baseline models and evaluation	You train your first regression model and report R^2 , MAE, RMSE, Median Absolute Error, and MDAPE.
Week 4	Model improvement and cross-validation	You improve the pipeline, understand cross-validation, and compare model performance rigorously.
Week 5	XGBoost and final predictor	You train an XGBoost model and package a simple prediction workflow.

Learning Outcomes

- Articulate a mental model of what drives California residential sale prices.
- Load and combine sold-property CSV files with pandas.
- Filter the data to single-family residences.
- Understand the difference between the target column and input features.
- Apply a repeatable decision framework to include or exclude any column.
- Avoid target leakage and explain why some features are not safe to use.
- Explore distributions, outliers, and feature relationships with matplotlib.
- Train simple baseline models before moving to stronger tree-based models.
- Report R^2 , MAE, RMSE, Median Absolute Error, and MDAPE, and understand each metric.
- Understand cross-validation and why a single train/test split has limitations.
- Train and evaluate an XGBoost regressor.
- Create a simple script that predicts ClosePrice for a California single-family property.

Data Files and Train / Test Split

You will be provided with seven monthly CSV files of California sold residential properties. Six files form the training set; the seventh is held out as the final test set. Do not use the test set during model development.

File	Role
CRMLSSold202509.csv	Training
CRMLSSold202510.csv	Training
CRMLSSold202511.csv	Training
CRMLSSold202512.csv	Training
CRMLSSold202601.csv	Training
CRMLSSold202602.csv	Training
CRMLSSold202603.csv	FINAL TEST SET — hold out until Week 5

Why hold out a separate test set?

A held-out test set simulates how the model will perform on data it has never seen. If you evaluate on data that influenced your modeling decisions, your reported metrics will be optimistic and misleading. Treat CRMLSSold202603.csv as locked until you are done building your model.

How to Think About House Prices

Before writing any code, it helps to ask a simple question: if you were a buyer, what would you actually pay more for?

The answer is almost always some combination of three things:

- Where is it? Location is consistently the strongest driver of price. Two identical houses sell for very different amounts depending on the city, ZIP code, and neighborhood. This is why `zip_median_price` and `city_median_price` end up being among the most predictive features in the model.
- How big is it and what does it have? Physical size (living area, lot size) and amenities (bedrooms, bathrooms, garage) drive willingness to pay directly. A buyer is largely paying for usable space.
- What condition is it in? Year built is a proxy for age and renovation cycles. Newer homes tend to command a premium; very old homes may have either a premium (historic character) or a discount (deferred maintenance).

That mental model maps directly to the feature groups in this project:

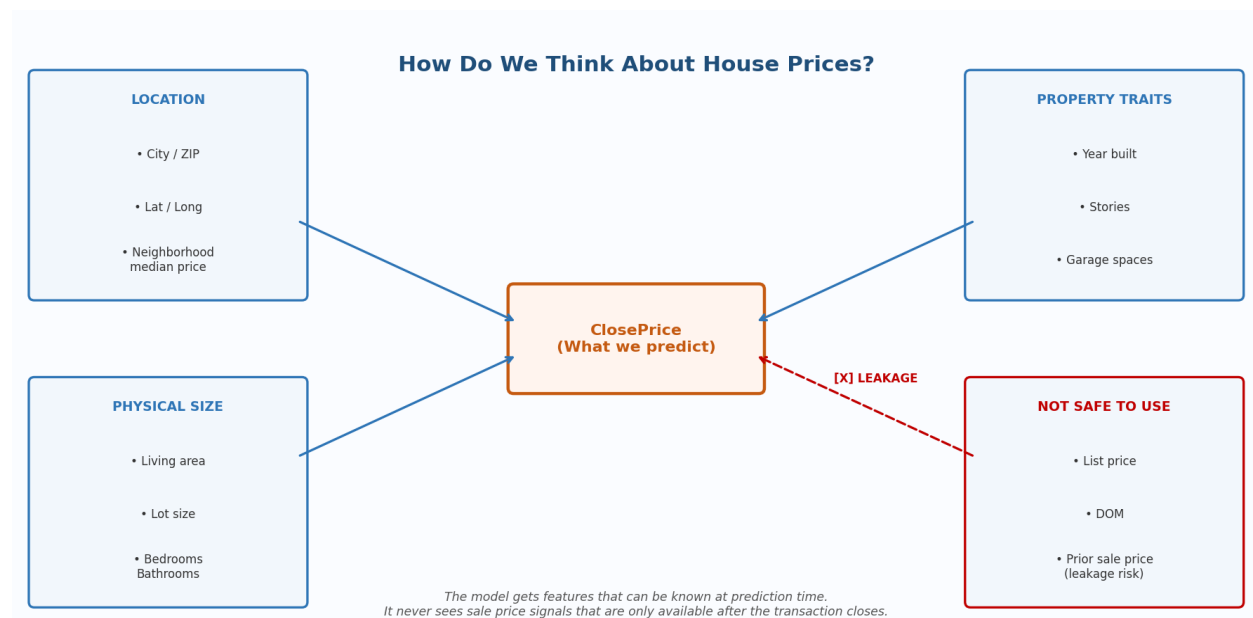


Figure 1: The four driver clusters that feed into `ClosePrice`. Only the three safe clusters are used as model inputs.

The fourth cluster in the diagram — labeled 'Not Safe to Use' — contains columns that are correlated with `ClosePrice` but cannot be known at prediction time, or that are derived from the sale itself. Using them would be target leakage: the model would appear to work on training data but fail in production because those values would not exist yet.

The key question to ask about any feature

Could I know this value if I were standing in front of the house before it sold? If yes, it is potentially safe. If no, or if it depends on the sale outcome, it is leakage.

Keep this mental model in mind throughout the program. When you are deciding which features to include, when you are debugging a bad prediction, or when you are explaining the model to someone else, this picture is the foundation.

Important Framing for This Project

The goal of this training is to predict sale price (ClosePrice) for California single-family homes using historical sold data. The model should rely on features that can be known at prediction time. That means we should prefer property characteristics, location variables, and engineered neighborhood signals. We should not use anything that directly reveals the final sale price or depends on information only known after the sale.

Week 0: Local Setup

Week 0 is not optional. Many beginners lose time because the machine learning code is fine but the local environment is broken. This week is only about setup, file loading, and making sure you can successfully run simple code.

Option A: Spyder on your computer

- Install Anaconda or Miniconda if it is not already installed.
- Open Anaconda Navigator and launch Spyder.
- Create a project folder such as `idx_ml_training` and place the CSV files inside a data subfolder.
- Inside Spyder, confirm you know where your current working directory is.

Recommended folder structure

```
idx_ml_training/  
├── data/  
│   ├── CRMLSSold202509.csv  
│   ├── CRMLSSold202510.csv  
│   ├── CRMLSSold202511.csv  
│   ├── CRMLSSold202512.csv  
│   ├── CRMLSSold202601.csv  
│   ├── CRMLSSold202602.csv  
│   └── CRMLSSold202603.csv  <-- HOLD OUT: final test set  
├── scripts/  
├── outputs/  
│   ├── cleaned_data/  
│   ├── metrics/  
│   └── models/  
└── README.txt
```

Option B: Google Colab

- Open a new Google Colab notebook.
- Upload the CSV files manually or mount Google Drive.
- Keep all file paths together near the top of the notebook.
- Run one cell at a time and make sure each cell finishes before moving on.

First imports and install

```
import pandas as pd  
import numpy as np  
from pathlib import Path  
  
# Run in Anaconda Prompt or terminal if needed:  
# pip install pandas numpy scikit-learn xgboost matplotlib joblib
```

How to confirm your files exist

```
from pathlib import Path

data_dir = Path('data')
training_files = [
    'CRMLSSold202509.csv', 'CRMLSSold202510.csv',
    'CRMLSSold202511.csv', 'CRMLSSold202512.csv',
    'CRMLSSold202601.csv', 'CRMLSSold202602.csv',
]
test_file = 'CRMLSSold202603.csv'

for name in training_files + [test_file]:
    path = data_dir / name
    print(name, 'exists:', path.exists())
```

Week 0 deliverable

You should be able to open Spyder or Colab, import pandas, load one CSV, print the shape, and list the columns. If that does not work yet, do not move to Week 1.

Week 1: Loading Files, Schema, and Cleaning

Now that the environment works, the next task is to combine the six training files into one training table. This week also establishes that data science begins with data quality, not with fancy models.

Load all training files and combine

```
import pandas as pd
from pathlib import Path

data_dir = Path('data')
training_files = [
    data_dir / 'CRMLSold202509.csv',
    data_dir / 'CRMLSold202510.csv',
    data_dir / 'CRMLSold202511.csv',
    data_dir / 'CRMLSold202512.csv',
    data_dir / 'CRMLSold202601.csv',
    data_dir / 'CRMLSold202602.csv',
]

dfs = [pd.read_csv(f) for f in training_files]
df = pd.concat(dfs, ignore_index=True)
print('Combined training shape:', df.shape)
```

Filter to the right housing subset

```
df = df[
    (df['PropertyType'] == 'Residential') &
    (df['PropertySubType'] == 'SingleFamilyResidence')
].copy()
print('After filter:', df.shape)
```

Initial inspection

```
print(df.dtypes)
print(df.describe())
print(df.isnull().sum().sort_values(ascending=False).head(20))
```

Convert numeric columns and remove invalid rows

```
numeric_cols = [
    'ClosePrice', 'BedroomsTotal', 'BathroomsTotalInteger', 'LivingArea',
    'LotSizeSquareFeet', 'YearBuilt', 'GarageSpaces', 'Stories',
    'Latitude', 'Longitude'
]

for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')
```

```
df = df[df['ClosePrice'] > 0].copy()
df = df[df['LivingArea'] > 0].copy()
df = df.dropna(subset=['ClosePrice', 'BedroomsTotal',
    'BathroomsTotalInteger', 'LivingArea', 'PostalCode']).copy()
```

Week 1 deliverable

A single combined DataFrame from the six training CSVs, filtered to single-family residences, with numeric columns converted and invalid rows removed. Print the row count and column list.

Week 2: Feature Engineering and Visualization

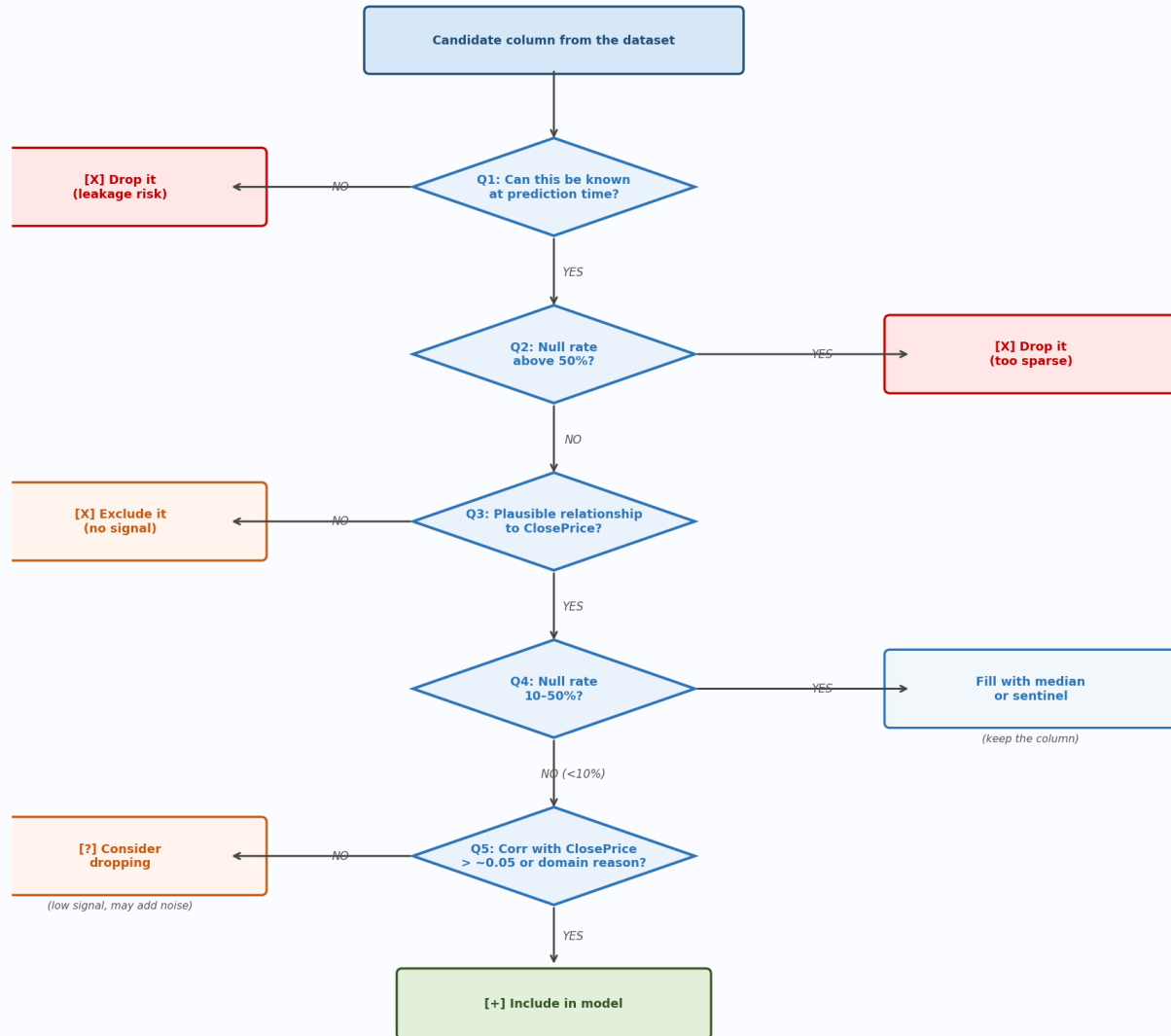
This is one of the most important weeks. You will decide which columns to include as model inputs, justify those choices, handle missing values deliberately, and explore the data visually before training anything.

Part A: Feature Selection Decision Framework

A good feature satisfies specific, testable conditions. Rather than choosing features by instinct, use the same five-question framework every time — for every column you consider, on this project and on any future project.

Feature Selection Decision Framework

Ask these four questions for every column you are considering.



Reminder: group-based features (zip_median_price, city_median_price) pass all five questions --
but must be computed from training data only to avoid leakage (see Week 2, Part C).

Figure 2: Apply these five questions to every candidate column. The same framework works on any dataset.

Walk through the five questions for a few example columns to see the framework in action:

Column	Q1: Known at prediction time?	Q2: Null > 50%?	Q3: Plausible signal?	Q4: Null 10-50%?	Decision
LivingArea	Yes	No	Yes — size is a primary driver	No (very low missingness; verify in dataset)	Include
zip_median_price	Yes (computed from training)	No	Yes — strongest location signal	No	Include (train split only)
ListPrice	No — set during active listing	N/A	N/A	N/A	Drop: leakage
LotSizeSquareFeet	Yes	No	Yes — land value	Yes (moderate missingness; verify exact %)	Fill median or 0 depending on context; include
AssociationFee	Yes	Yes (moderate missingness ~30%; verify on your dataset)	Yes — HOA affects buyer cost	Yes (~30%)	Include with imputation or feature engineering

A note on borderline columns

Some columns will sit on the edge of the framework — meaningful signal but moderate missingness, or plausible but inconsistent across datasets. Document your reasoning. A thoughtful decision you can explain is always better than a gut choice you cannot defend.

In real MLS data, it is common for some features to vary significantly in availability depending on the month, region, or data source. For example, in one sample dataset used in this program, `AssociationFee` is missing in approximately 30% of rows — not high enough to automatically drop, but not low enough to ignore. This makes it a *borderline feature* that requires judgment.

Note on missing value percentages

Missing value percentages should not be assumed — they must be measured directly from your dataset.

For example:

- `AssociationFee` may have **moderate missingness (~30%)** and could still be useful with proper handling

Because MLS data structures can change across time and feeds, the same column may behave very differently in another file.

How to handle this in practice

The key principle is not the exact percentage, but the decision framework:

- **High missingness (>50%)** → consider dropping unless the feature is very important
- **Moderate missingness (10–50%)** → consider imputation or feature engineering
- **Low missingness (<10%)** → typically safe to include

For moderate-missing features like `AssociationFee`, you might:

- Fill missing values with 0 if null implies “no HOA”
- Create a binary feature (e.g., `has_hoa = 0/1`)
- Keep the original column and evaluate its importance in the model

Example: compute missing rates

```
null_rates = df.isnull().mean().sort_values(ascending=False)
print((null_rates * 100).round(2))
```

Part B: Chosen Features and Justifications

Feature	Why include it	Watch out for
<code>BedroomsTotal</code>	Strong direct driver of price; buyers care about bedroom count.	Extreme values (0 or 20+) are likely data errors.
<code>BathroomsTotalInteger</code>	Similar logic to bedrooms; adds independent signal.	May be missing or stored as a float.
<code>LivingArea</code>	Square footage is one of the strongest price predictors.	Values below 200 or above 15,000 are suspect.
<code>LotSizeSquareFeet</code>	Larger lots tend to command higher prices in many markets.	Many townhomes have zero or null lot size — fill with 0 or median.
<code>YearBuilt</code>	Newer homes often sell at a premium; captures renovation cycles.	Future years or pre-1800 values are data errors.

Feature	Why include it	Watch out for
GarageSpaces	Desirable amenity; proxy for property quality.	Null often means no garage, not unknown — fill with 0.
Stories	Captures building style and floor plan.	Sometimes stored as text ('Two Story'); convert to numeric.
Latitude / Longitude	Direct location signal without needing to encode strings.	Must be plausible for California (lat ~32-42, lon ~-124 to -114).
zip_median_price	Neighborhood price level; one of the strongest location signals.	Must be computed from training data only to avoid leakage.
city_median_price	City-level signal complements ZIP; smooths thin ZIP markets.	Same leakage caution applies.

Part C: Handling Missing Values

Do not simply drop every row with a null. Make a deliberate decision for each column using the framework above.

Rule	When to apply it
Drop the row	The column is required for the model and cannot be estimated (e.g., ClosePrice, LivingArea, PostalCode).
Fill with median	The column is useful but missing for fewer than 50% of rows and the median is a reasonable stand-in (e.g., LotSizeSquareFeet, YearBuilt).
Fill with a sentinel value	Null genuinely means 'none' (e.g., GarageSpaces null → fill with 0).
Drop the column	More than 50% of rows are null and the feature is not critical. Mostly-empty columns add noise.
Investigate further	Null rate is high but the column seems important. Check whether null is correlated with property type or location before deciding.

```
# Check null rates for all columns
null_rates = df.isnull().mean().sort_values(ascending=False)
print(null_rates[null_rates > 0])

# Sentinel fill: null GarageSpaces means no garage
df['GarageSpaces'] = df['GarageSpaces'].fillna(0)

# Median fill for columns with modest null rates
for col in ['LotSizeSquareFeet', 'YearBuilt', 'Stories', 'Latitude', 'Longitude']:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].median())
```

Part D: Engineered Location Features

Median price by ZIP and city encode neighborhood desirability in a single number. The critical rule is to compute these medians using the training data only.

```
df['PostalCode5'] = df['PostalCode'].astype(str).str.slice(0, 5)
df['City'] = df['City'].astype(str)

# These use the full training DataFrame -- do NOT include test data here
df['zip_median_price'] =
df.groupby('PostalCode5')['ClosePrice'].transform('median')
df['city_median_price'] = df.groupby('City')['ClosePrice'].transform('median')
```

Leakage warning: group-based features

If you compute `zip_median_price` on the full dataset (including the test set), you are leaking future sale prices into the training process. Always compute group medians on the training split, then map those values into the validation or test set.

Part E: Exploratory Visualization

Before training any model, look at the data. Visualization helps you catch outliers, understand distributions, and develop intuition about which features matter most.

1. Distribution of ClosePrice

```
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.hist(df['ClosePrice'], bins=80, color='steelblue', edgecolor='white')
plt.xlabel('ClosePrice')
plt.title('ClosePrice distribution (raw)')

plt.subplot(1, 2, 2)
plt.hist(np.log1p(df['ClosePrice']), bins=80, color='darkorange',
edgecolor='white')
plt.xlabel('log(ClosePrice + 1)')
plt.title('ClosePrice distribution (log scale)')

plt.tight_layout()
plt.savefig('outputs/price_distribution.png', dpi=120)
plt.show()
```

If the raw distribution has a long right tail, log-transforming the target is worth considering in Week 4.

2. Scatter plots: feature vs. ClosePrice

```
features_to_plot = ['LivingArea', 'BedroomsTotal', 'YearBuilt', 'zip_median_price']
```

```

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for ax, feat in zip(axes.flat, features_to_plot):
    ax.scatter(df[feat], df['ClosePrice'], alpha=0.2, s=5, color='steelblue')
    ax.set_xlabel(feat)
    ax.set_ylabel('ClosePrice')
    ax.set_title(f'{feat} vs ClosePrice')

plt.tight_layout()
plt.savefig('outputs/feature_scatter.png', dpi=120)
plt.show()

```

Look for: a clear upward trend (good), a flat cloud (weak feature), or suspicious clusters and gaps (data quality issues).

3. Missing value bar chart

```

null_rates = df[feature_cols].isnull().mean().sort_values()
null_rates.plot(kind='barh', figsize=(8, 5), color='tomato')
plt.xlabel('Null fraction')
plt.title('Missing value rate by feature')
plt.tight_layout()
plt.savefig('outputs/null_rates.png', dpi=120)
plt.show()

```

4. Correlation matrix

```

corr = df[feature_cols + ['ClosePrice']].corr()
fig, ax = plt.subplots(figsize=(10, 8))
im = ax.imshow(corr, cmap='coolwarm', vmin=-1, vmax=1)
plt.colorbar(im, ax=ax)
ax.set_xticks(range(len(corr.columns)))
ax.set_yticks(range(len(corr.columns)))
ax.set_xticklabels(corr.columns, rotation=45, ha='right', fontsize=9)
ax.set_yticklabels(corr.columns, fontsize=9)
plt.title('Feature correlation matrix')
plt.tight_layout()
plt.savefig('outputs/correlation_matrix.png', dpi=120)
plt.show()

```

Week 2 deliverable

A feature list with a written justification for each feature, applying the five-question framework. A missing-value decision for every column. Saved plots: price distribution, at least two feature scatter plots, null rate chart. A modeling table with no missing values in the selected inputs.

Week 3: Baseline Models and Evaluation Metrics

This week focuses on training your first models and measuring them carefully. A beginner often asks 'Is my model good?' The honest answer is that no single number fully answers that question.

Build the modeling table and split

```
from sklearn.model_selection import train_test_split

feature_cols = [
    'BedroomsTotal', 'BathroomsTotalInteger', 'LivingArea', 'LotSizeSquareFeet',
    'YearBuilt', 'GarageSpaces', 'Stories', 'Latitude', 'Longitude',
    'zip_median_price', 'city_median_price'
]

model_df = df[feature_cols + ['ClosePrice']].dropna().copy()
X = model_df[feature_cols]
y = model_df['ClosePrice']

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.20, random_state=42
)
```

Recommended metrics

Metric	What it tells you
R ²	How much variation in ClosePrice is explained by the model. 1.0 is perfect; 0 means no better than predicting the mean.
MAE	Average absolute dollar error. Easy to explain: 'on average we are off by \$X'.
RMSE	Gives extra penalty to large mistakes. Higher than MAE means the model has some very big misses.
Median Absolute Error	Typical absolute error using the median. More robust to outliers than MAE.
MDAPE	Median absolute percentage error. Good for explaining typical percentage miss while being robust to extreme California prices.

Metric functions

```
import numpy as np
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error,
median_absolute_error

def mdape(y_true, y_pred):
    y_true = np.array(y_true)
```

```

y_pred = np.array(y_pred)
mask = y_true != 0
ape = np.abs((y_true[mask] - y_pred[mask]) / y_true[mask]) * 100
return np.median(ape)

def regression_report(y_true, y_pred):
    return {
        'R2': r2_score(y_true, y_pred),
        'MAE': mean_absolute_error(y_true, y_pred),
        'RMSE': mean_squared_error(y_true, y_pred, squared=False),
        'MedianAbsoluteError': median_absolute_error(y_true, y_pred),
        'MDAPE': mdape(y_true, y_pred),
    }

```

Train baseline models

```

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor

lr = LinearRegression()
lr.fit(X_train, y_train)
print('Linear Regression:', regression_report(y_val, lr.predict(X_val)))

rf = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
print('Random Forest:', regression_report(y_val, rf.predict(X_val)))

```

How to interpret the metrics together

- If MAE is low but RMSE is much higher, the model has some very large misses.
- If MDAPE looks reasonable but MAE is still high, the model is doing proportionally well but missing by large dollar amounts on expensive homes.
- If R^2 improves meaningfully from the linear baseline to more complex models, the new model is capturing more structure in the data.
- Never judge a housing model on a single metric alone.

A note on cross-validation

The train/validation split above uses a single 80/20 split. This is a reasonable starting point. However, results depend on which rows happened to end up in the validation set. A more robust approach is k-fold cross-validation, where you split the training data into k equal parts, train on k-1 parts, and evaluate on the remaining part, rotating k times. The average metric across folds is more reliable than a single split.

```

from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(lr, X, y, cv=5, scoring='neg_mean_absolute_error')
print('CV MAE (mean):', -cv_scores.mean())
print('CV MAE (std): ', cv_scores.std())
# A large std means performance varies a lot across folds -- investigate why

```

Cross-validation and the final test set

Cross-validation is performed on the training data only. The final test set (CRMLSSold202603.csv) is still held out. Cross-validation helps you compare models and tune hyperparameters without peeking at the test set.

Week 3 deliverable

A printed metric summary for the linear regression baseline. A printed metric summary for at least one additional model. A short note explaining what changed and whether performance improved.

Week 4: Model Improvement

This week focuses on improving the pipeline before moving to XGBoost. Common improvements include transforming the target, adding stronger location features, and cleaning up edge cases.

Ideas to try

- Log-transform the target using `np.log1p(ClosePrice)` if the price distribution is heavily right-skewed. Back-transform with `np.expm1()` before computing metrics.
- Clip or remove impossible structural values (bedrooms > 15, living area > 15,000, year built < 1850).
- Add stronger location features: county-level medians or distance to city center if latitude/longitude are available.
- Compute group-based features on the training split only, then map into validation.

Proper group-feature mapping (no leakage)

```
# Step 1: compute medians on training split only
zip_map = X_train.join(y_train).groupby('PostalCode5')['ClosePrice'].median()
city_map = X_train.join(y_train).groupby('City')['ClosePrice'].median()

# Step 2: apply to validation (and later, to test)
global_median = y_train.median()
X_val['zip_median_price'] =
X_val['PostalCode5'].map(zip_map).fillna(global_median)
X_val['city_median_price'] = X_val['City'].map(city_map).fillna(global_median)
```

Visualizing residuals

```
import matplotlib.pyplot as plt

plt.figure(figsize=(7, 7))
plt.scatter(y_val, rf.predict(X_val), alpha=0.3, s=8, color='steelblue')
plt.plot([y_val.min(), y_val.max()], [y_val.min(), y_val.max()], 'r--', lw=1.5)
plt.xlabel('Actual ClosePrice')
plt.ylabel('Predicted ClosePrice')
plt.title('Actual vs. Predicted (validation set)')
plt.tight_layout()
plt.savefig('outputs/actual_vs_predicted.png', dpi=120)
plt.show()
```

Week 4 deliverable

At least one improvement to the pipeline. A comparison table of metric values before and after. A saved actual-vs-predicted scatter plot.

Week 5: XGBoost and Final Prediction Workflow

By the time you reach this week, the data should already be cleaned, the feature table should already exist, and at least one simpler baseline should already have been evaluated.

Why XGBoost

XGBoost is a gradient-boosted tree model that often performs very well on structured tabular data. Real estate datasets frequently contain nonlinear relationships and interactions between features that tree-based boosting captures better than a linear model.

Train XGBoost

```
from xgboost import XGBRegressor

xgb_model = XGBRegressor(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.85,
    colsample_bytree=0.85,
    objective='reg:squarederror',
    random_state=42
)

xgb_model.fit(X_train, y_train)
xgb_preds = xgb_model.predict(X_val)
print('XGBoost (validation):', regression_report(y_val, xgb_preds))
```

Feature importance

```
importance_df = pd.DataFrame({
    'feature': X_train.columns,
    'importance': xgb_model.feature_importances_
}).sort_values('importance', ascending=False)

importance_df.head(12).plot(kind='barh', x='feature', y='importance',
                             figsize=(8, 6), legend=False, color='steelblue')

plt.xlabel('Importance')
plt.title('XGBoost feature importance')
plt.tight_layout()
plt.savefig('outputs/feature_importance.png', dpi=120)
plt.show()
```

Evaluate on the final test set

Only run this step once, after all modeling decisions are final. Load CRMLSSold202603.csv, apply the exact same cleaning and feature logic used on the training data, then evaluate.

```
df_test = pd.read_csv(data_dir / 'CRMLSSold202603.csv')
df_test = df_test[
```

```

(df_test['PropertyType'] == 'Residential') &
(df_test['PropertySubType'] == 'SingleFamilyResidence')
].copy()

for col in numeric_cols:
    if col in df_test.columns:
        df_test[col] = pd.to_numeric(df_test[col], errors='coerce')

df_test['GarageSpaces'] = df_test['GarageSpaces'].fillna(0)
for col in ['LotSizeSquareFeet', 'YearBuilt', 'Stories', 'Latitude', 'Longitude']:
    if col in df_test.columns:
        df_test[col] = df_test[col].fillna(df[col].median()) # use training median

df_test['PostalCode5'] = df_test['PostalCode'].astype(str).str.slice(0, 5)
df_test['City'] = df_test['City'].astype(str)
df_test['zip_median_price'] =
df_test['PostalCode5'].map(zip_map).fillna(global_median)
df_test['city_median_price'] = df_test['City'].map(city_map).fillna(global_median)

X_test = df_test[feature_cols].dropna()
y_test = df_test.loc[X_test.index, 'ClosePrice']

test_preds = xgb_model.predict(X_test)
print('Final test set metrics:', regression_report(y_test, test_preds))

```

Final prediction function

```

def predict_close_price(model, input_dict, feature_cols):
    input_df = pd.DataFrame([input_dict])
    for col in feature_cols:
        if col not in input_df.columns:
            input_df[col] = 0
    return model.predict(input_df[feature_cols])[0]

sample_property = {
    'BedroomsTotal': 4, 'BathroomsTotalInteger': 3,
    'LivingArea': 2400, 'LotSizeSquareFeet': 6500,
    'YearBuilt': 1998, 'GarageSpaces': 2,
    'Stories': 2, 'Latitude': 34.121, 'Longitude': -117.891,
    'zip_median_price': 925000, 'city_median_price': 890000
}

print('Predicted ClosePrice:', predict_close_price(xgb_model, sample_property,
feature_cols))

```

Week 5 deliverable

A trained XGBoost model. A side-by-side comparison of baseline and XGBoost metrics on the validation set. Final test set metrics from CRMLSold202603.csv. A feature importance chart. A final prediction function.

Appendix A: Complete Starter Pipeline

The following script shows a simple end-to-end starting point using the six training files.

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (r2_score, mean_absolute_error,
                             mean_squared_error, median_absolute_error)
from xgboost import XGBRegressor

def mdape(y_true, y_pred):
    y_true, y_pred = np.array(y_true), np.array(y_pred)
    mask = y_true != 0
    return np.median(np.abs((y_true[mask]-y_pred[mask])/y_true[mask])*100)

def regression_report(y_true, y_pred):
    return {'R2': r2_score(y_true, y_pred),
            'MAE': mean_absolute_error(y_true, y_pred),
            'RMSE': mean_squared_error(y_true, y_pred, squared=False),
            'MedianAbsoluteError': median_absolute_error(y_true, y_pred),
            'MDAPE': mdape(y_true, y_pred)}

data_dir = Path('data')
training_files = [data_dir/f'CRMLSSold2025{m}.csv' for m in
                  ['09', '10', '11', '12']] + [data_dir/f'CRMLSSold202601.csv',
                  data_dir/'CRMLSSold202602.csv']

df = pd.concat([pd.read_csv(f) for f in training_files], ignore_index=True)
df = df[(df['PropertyType']=='Residential') &
        (df['PropertySubType']=='SingleFamilyResidence')].copy()

numeric_cols = ['ClosePrice', 'BedroomsTotal', 'BathroomsTotalInteger', 'LivingArea',
                'LotSizeSquareFeet', 'YearBuilt', 'GarageSpaces', 'Stories', 'Latitude', 'Longitude']
for col in numeric_cols:
    if col in df.columns: df[col] = pd.to_numeric(df[col], errors='coerce')

df = df[(df['ClosePrice']>0) & (df['LivingArea']>0)]
df = df.dropna(subset=['ClosePrice', 'BedroomsTotal', 'BathroomsTotalInteger',
                      'LivingArea', 'PostalCode']).copy()
df['GarageSpaces'] = df['GarageSpaces'].fillna(0)
for col in ['LotSizeSquareFeet', 'YearBuilt', 'Stories', 'Latitude', 'Longitude']:
    if col in df.columns: df[col] = df[col].fillna(df[col].median())

df['PostalCode5'] = df['PostalCode'].astype(str).str.slice(0,5)
df['City'] = df['City'].astype(str)
df['zip_median_price'] =
df.groupby('PostalCode5')['ClosePrice'].transform('median')
df['city_median_price'] = df.groupby('City')['ClosePrice'].transform('median')

feature_cols =
['BedroomsTotal', 'BathroomsTotalInteger', 'LivingArea', 'LotSizeSquareFeet',
 'YearBuilt', 'GarageSpaces', 'Stories', 'Latitude', 'Longitude',
 'zip_median_price', 'city_median_price']

model_df = df[feature_cols+['ClosePrice']].dropna().copy()
X, y = model_df[feature_cols], model_df['ClosePrice']
```

```
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.20,  
random_state=42)
```

```
lr = LinearRegression().fit(X_train, y_train)  
print('Baseline:', regression_report(y_val, lr.predict(X_val)))
```

```
xgb = XGBRegressor(n_estimators=500, learning_rate=0.05, max_depth=6,  
    subsample=0.85, colsample_bytree=0.85,  
    objective='reg:squarederror', random_state=42)  
xgb.fit(X_train, y_train)  
print('XGBoost:', regression_report(y_val, xgb.predict(X_val)))
```

Appendix B: Common Mistakes and Debugging Checklist

- The file path is wrong, so pandas cannot find the CSV.
- You loaded CRMLSSold202603.csv into the training set. That file is the final test set.
- You filtered to PropertyType but forgot PropertySubType = SingleFamilyResidence.
- The target column ClosePrice still has missing values after cleaning.
- A numeric column is stored as text and needs pd.to_numeric.
- You used a feature that cannot be known at prediction time (target leakage).
- The model was trained before missing values were handled.
- ZIP or city median features were computed using the full dataset instead of the training split only.
- You dropped a column without checking its null rate first.
- Metrics were reported for one model but not compared against a baseline.
- Predictions were never back-transformed after log-price modeling.

Final checklist

Question	Yes / No
Can you explain the mental model of what drives California home prices?	
Can you load all six training CSV files?	
Did you hold out CRMLSSold202603.csv as the test set?	
Did you filter to Residential / SingleFamilyResidence?	
Did you apply the five-question framework to every feature?	
Did you review null rates and make a deliberate decision for each column?	
Did you save and review at least three exploratory plots?	
Did you train a baseline model and record its metrics?	
Did you report R ² , MAE, RMSE, Median Absolute Error, and MDAPE?	
Did you train an XGBoost model and compare it to the baseline?	
Did you evaluate the final model on the held-out test set exactly once?	
Can you explain how a street address becomes model features?	

Final note for trainees

You do not need the perfect model in Week 5. The most important outcome is that you understand the workflow end to end and can explain what you built, why you chose your features, how you evaluated the model, and where you would improve it next.